APB NUMBER THEORETIC TRANSFORM CORE

Radix-2 Forward NTT Accelerator with Internal Coefficient Memory

PURPOSE

This document defines the externally visible behavior, functional architecture, programming model, clock and reset requirements, and verification intent of the APB Number Theoretic Transform Core. It is the integration baseline for RTL, verification, synthesis, and SoC teams.

DOCUMENT TYPE

Hardware Architecture Specification (HAS)

DOCUMENT ID

HAS-APB-NTT-001

REVISION

1.0

STATUS

Detailed engineering specification

DATE

18 July 2026

CLASSIFICATION

Internal technical documentation

TABLE OF CONTENTS

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

REVISION HISTORY

Table 1: Document revision history

Revision	Date	Author	Description
B	18 July 2026	Brian Bui	Complete RTL-aligned architecture and programming specification

Solution Brief

Description

The APB NTT core is a synthesizable, single-clock accelerator that executes an in-place radix-2 decimation-in-time forward cyclic Number Theoretic Transform. Software loads coefficients through an indexed APB window, starts the transform, observes completion or error status, and reads the result from the same memory. The integration top is apb_ntt_wrapper.

Architecture

The integration top combines a qualified 32-bit APB programming interface, an indexed coefficient window, synchronous dual-read/dual-write core memory ports, an eight-state controller, a stage-aware address generator, a generated twiddle ROM, and a combinational modular butterfly datapath. With the integrated memory handshake, the core sustains one in-place butterfly update every five PCLK cycles after initialization.

Key Features

- 32-bit APB subordinate interface with byte strobes, immediate PREADY, PSLVERR, and IRQ.
- Parameterized transform length, prime modulus, root, coefficient width, and address width.
- Radix-2 decimation-in-time forward cyclic NTT with in-place dual-port memory updates.
- Combinational modular multiplication, Barrett reduction, addition, and subtraction.
- Generated twiddle ROM containing powers of the configured root.
- Sticky DONE and ERROR status with software CLEAR control.
- Protection against coefficient access and repeated START while BUSY.
- Synopsys VCS/UVM and SpyGlass compile, lint, CDC, and RDC flows.

Target Applications

- Lattice-cryptography and polynomial-arithmetic research.
- FPGA or ASIC algorithm acceleration prototypes.
- Embedded coprocessors using a memory-mapped NTT primitive.
- Architecture exploration for configurable modular arithmetic engines.

Integration note

Technical information is subject to implementation review. System-level safety, security, entropy, retention, and cryptographic compliance remain the integrator's responsibility unless explicitly stated.

1 Product Overview

The APB NTT core is a synthesizable, single-clock accelerator that executes an in-place radix-2 decimation-in-time forward cyclic Number Theoretic Transform. Software loads coefficients through an indexed APB window, starts the transform, observes completion or error status, and reads the result from the same memory. The integration top is apb_ntt_wrapper.

1.1 Scope

This specification defines the externally observable behavior and implementation contract of apb_ntt_wrapper: APB transfers, coefficient memory access, transform algorithm and ordering, controller sequencing, arithmetic assumptions, reset behavior, interrupt/error handling, parameters, software flow, and current verification evidence.

1.2 Design Objectives

- Provide a deterministic APB-controlled forward NTT execution path.
- Keep operand and result storage internal to a simple indexed software window.
- Produce canonical output coefficients in the range 0 through $Q - 1$.
- Make invalid commands and protected accesses visible through PSLVERR or sticky ERROR.
- Keep the RTL hierarchy and verification flow reusable across compatible parameter sets.

1.3 Target Applications

- Lattice-cryptography and polynomial-arithmetic research.
- FPGA or ASIC algorithm acceleration prototypes.
- Embedded coprocessors using a memory-mapped NTT primitive.
- Architecture exploration for configurable modular arithmetic engines.

1.4 Out of Scope

Inverse NTT, pointwise multiplication, DMA, streaming interfaces, cache coherency, zeroization, side-channel countermeasures, constant-time security certification, clock gating, scan/test integration, ECC/parity memory protection, and standard-specific negacyclic polynomial mapping are not implemented.

2 General Description

2.1 Features

- 32-bit APB subordinate interface with byte strobes, immediate PREADY, PSLVERR, and IRQ.
- Parameterized transform length, prime modulus, root, coefficient width, and address width.
- Radix-2 decimation-in-time forward cyclic NTT with in-place dual-port memory updates.
- Combinational modular multiplication, Barrett reduction, addition, and subtraction.
- Generated twiddle ROM containing powers of the configured root.
- Sticky DONE and ERROR status with software CLEAR control.
- Protection against coefficient access and repeated START while BUSY.
- Synopsys VCS/UVM and SpyGlass compile, lint, CDC, and RDC flows.

2.2 Standards Compliance

The programming interface follows the APB setup/access transfer model and exposes PSTRB and PSLVERR behavior associated with a modern 32-bit APB integration. The RTL has not been independently certified for AMBA protocol compliance. The transform is a generic cyclic NTT primitive and does not by itself implement or claim compliance with ML-KEM, ML-DSA, or another cryptographic algorithm standard.

2.3 Functional Description

The integration hierarchy is rooted at apb_ntt_wrapper. The apb_ntt_apb_if block decodes the four software-visible offsets, stores MODE and the coefficient index, produces one-cycle START and host-write pulses, implements sticky DONE/ERROR state, and qualifies PSLVERR. The apb_ntt_coefficient_memory block provides an asynchronous host read window and registered dual-operand reads for the transform core. Core writes have priority over a host write; the APB protection logic prevents a legal host data access while the core is BUSY, so the two clients do not intentionally write concurrently.

The apb_ntt_core block separates sequencing, address generation, and arithmetic. Its controller advances only on the memory read-valid and write-ready handshakes. The address generator maintains a stage counter and an $N/2$-entry butterfly counter. The datapath canonicalizes both fetched operands, captures the twiddle index with the operands, evaluates one modular butterfly combinationally, and returns two coefficients to the same memory addresses.

Let N be a power of two, Q the modulus, and $\omega = \text{ROOT} \bmod Q$ an element of exact order N. The conventional forward cyclic NTT is

$$X[k] = \sum_{n=0}^{N-1} x[n]\omega^{nk} \bmod Q, \qquad 0 \le k < N. \tag{1}$$

The RTL implements the iterative radix-2 decimation-in-time butterfly

$$t = b \cdot \omega^e \bmod Q, \tag{2}$$

$$y_0 = a + t \bmod Q, \tag{3}$$

$$y_1 = a - t \bmod Q. \tag{4}$$

For stage $s \in [0, \log_2 N - 1]$ and butterfly counter $p \in [0, N/2 - 1]$, the address generator computes

$$h = 2^s, \qquad g = \lfloor p/h \rfloor, \qquad j = p \bmod h, \tag{5}$$

$$A = 2hg + j, \qquad B = A + h, \qquad e = j \cdot 2^{\log_2 N - 1 - s}. \tag{6}$$

The core reads coefficients at A and B, reduces both into the canonical residue range, evaluates the butterfly using ω^e, and writes the two results back to the same addresses.

Coefficient ordering contract

This is a DIT schedule without an internal bit-reversal permutation. To obtain the conventional natural-order transform shown above, software shall load the natural input vector at bit-reversed coefficient addresses. Loading natural-order input directly computes the DIT network on that unpermuted sequence and is not equivalent to a simple natural-order forward NTT result.

2.3.1 Arithmetic Contract

The multiplier forms a $2\times$COEFF_WIDTH product and reduces it with a combinational Barrett reducer. The reducer uses $\mu = \lfloor 2^{2\lceil \log_2 Q \rceil}/Q \rfloor$ followed by one conditional subtraction. The modular adder likewise performs at most one subtraction of Q, while the subtractor adds Q on underflow. The contract therefore requires canonical butterfly operands below Q and reducer inputs within the single-correction bound. Raw host coefficients are stored without modification, then reduced when captured by the datapath. Every written transform result is a canonical residue in $[0, Q - 1]$.

2.3.2 Twiddle Generation

The twiddle ROM contains $N/2$ combinational constants. Entry i is generated at elaboration as ROOT^i mod Q. The selected root must have exact order N; the RTL does not dynamically validate this condition.

2.3.3 Controller Sequence

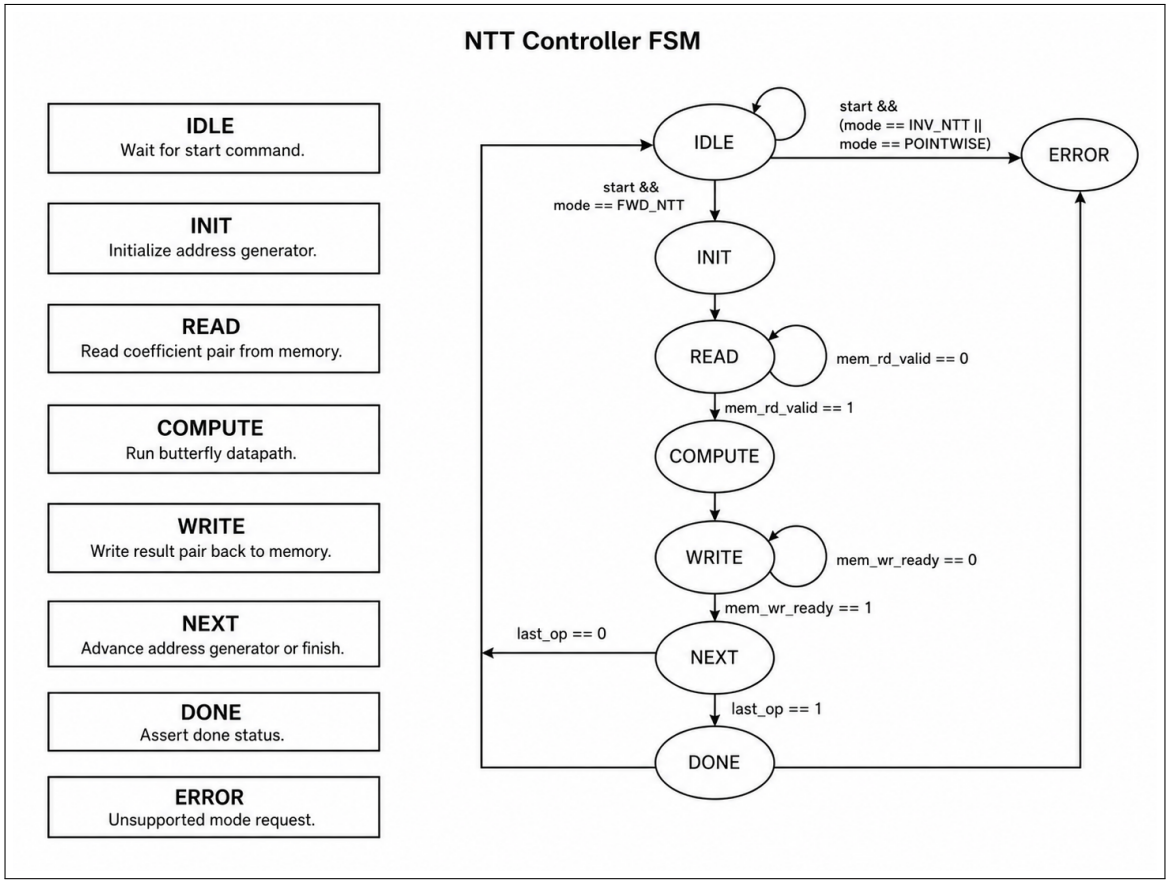

Figure 1: NTT controller state flow; Table 2 is normative

Table 2: NTT controller state behavior

State	BUSY	Primary action	Transition condition
IDLE	0	Wait for START.	Forward mode enters INIT; unsupported modes enter ERROR.
INIT	1	Clear stage and butterfly counters.	Unconditional transition to READ.
READ	1	Request both coefficient operands.	Memory read-valid enters COMPUTE.
COMPUTE	1	Evaluate the captured combinational butterfly.	Unconditional transition to WRITE.
WRITE	1	Write both results in place.	Memory write-ready enters NEXT.

State	BUSY	Primary action	Transition condition
NEXT	1	Test last operation and advance counters.	Last operation enters DONE; otherwise READ.
DONE	0	Pulse the internal done indication.	Unconditional transition to IDLE.
ERROR	0	Pulse the internal error indication.	Unconditional transition to IDLE.

2.3.4 Execution Latency

The integrated memory has one-cycle registered read-valid and permanently asserted write-ready. READ therefore occupies two cycles; COMPUTE, WRITE, and NEXT each occupy one. With

$$B = \frac{N}{2} \log_2 N \tag{7}$$

butterflies, BUSY is asserted for $1 + 5B$ PCLK cycles. Sticky DONE and IRQ become visible nominally $3 + 5B$ rising-edge intervals after completion of the APB START access.

Table 3: Nominal execution latency

Configuration	Butterflies	BUSY cycles	START-to-IRQ cycles
$N = 8$ UVM configuration	12	61	63
$N = 256$ default configuration	1024	5121	5123

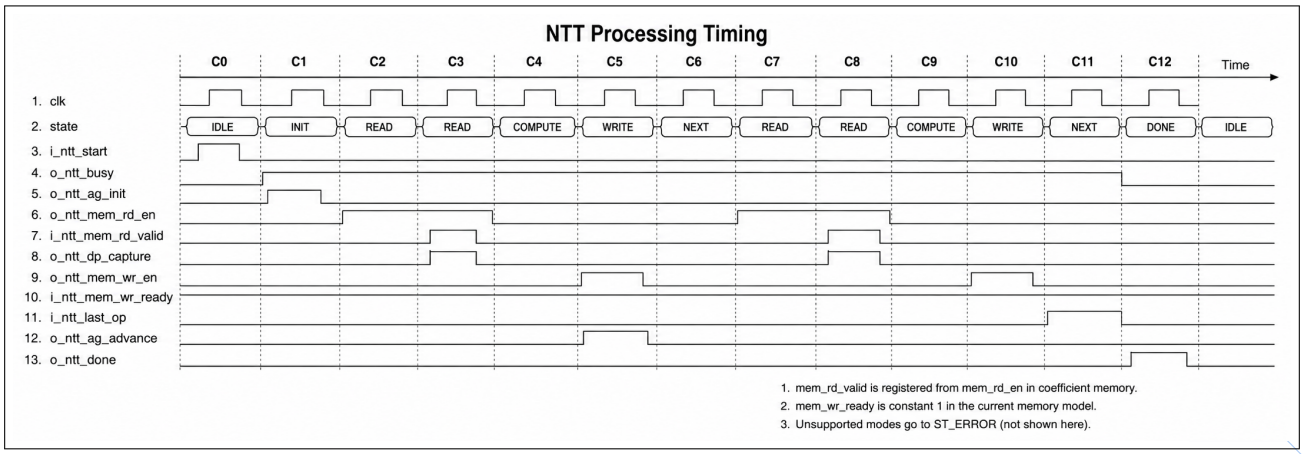

Figure 2: Integrated-memory NTT processing timing example

Figure 2 illustrates the registered read-valid response, permanently asserted write-ready response, and controller phase ordering. Cycle counts in Table 3 are normative; an implementation that replaces the integrated memory must preserve the handshake protocol and account for any additional READ or WRITE wait cycles.

2.4 Architecture

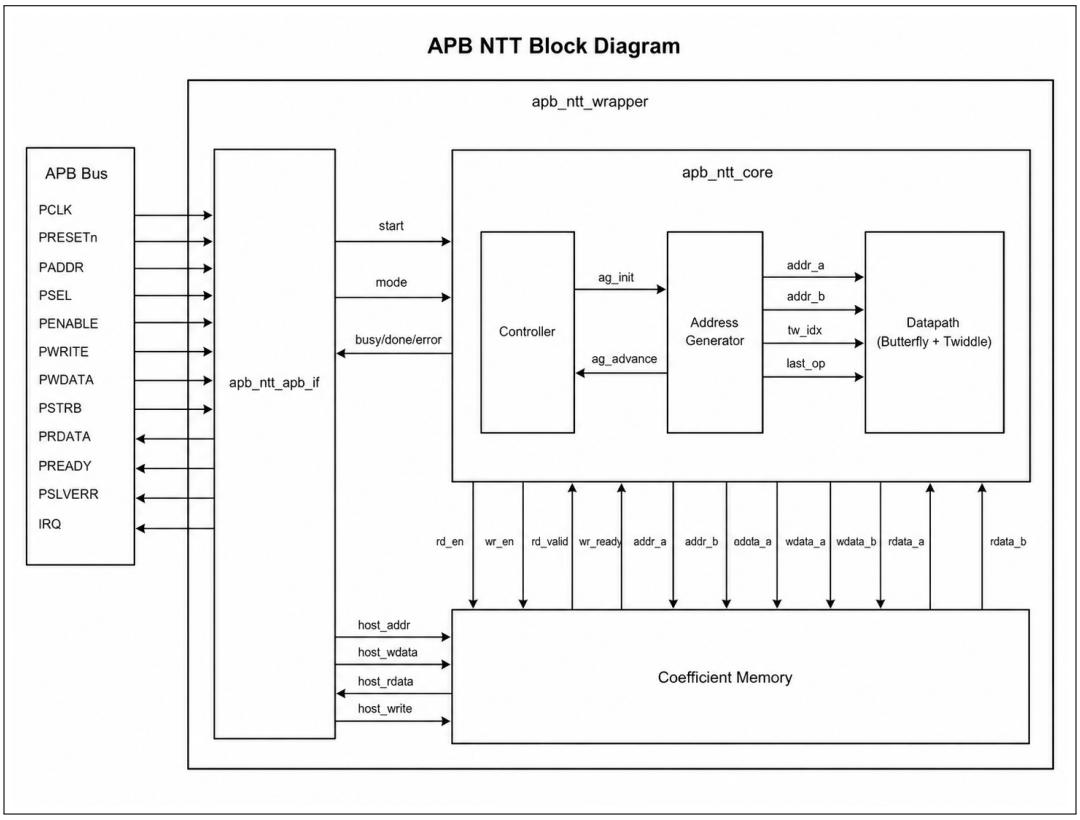

Figure 3: APB NTT functional architecture

2.5 Block Description

Table 4: Functional block responsibilities

Block	Responsibility	Implementation notes
apb_ntt_wrapper	Integration top	Connects the APB interface, coefficient memory, and transform core in one PCLK domain.

Block	Responsibility	Implementation notes
apb_ntt_apb_if	Programming model	Decodes four registers, applies PSTRB, generates command pulses, latches status, reports PSLVERR, and drives IRQ.
apb_ntt_coefficient_memory	Operand/result storage	Stores N coefficients, provides asynchronous host read, registered dual-core read, and dual-core write. Core write has priority over host write.
apb_ntt_core	Core integration	Connects the controller, address generator, and datapath through explicit memory handshakes.
apb_ntt_controller	Sequencing	Implements the eight-state execution and unsupported-mode error state machine.
apb_ntt_address_generator	Scheduling	Generates stage-dependent coefficient addresses, twiddle index, and last-operation flag.
apb_ntt_datapath	Arithmetic pipeline	Reduces and captures operands/twiddle index, selects a twiddle, and evaluates one butterfly.
apb_ntt_twiddle_rom	Constant generation	Elaborates $N/2$ powers of ROOT modulo Q as combinational constants.
apb_ntt_butterfly	Modular operation	Computes twiddle multiplication followed by modular sum and difference.

2.6 Integration Constraints

Use the ordered RTL file list with apb_ntt_wrapper as the integration top. Connect all ports to one PCLK/reset domain, propagate PSLVERR and IRQ to the system, load all coefficient locations before START, and ensure the selected parameter set satisfies the constraints in Section 5. Memory inference, timing closure, root selection, coefficient ordering, and end-to-end algorithm mapping remain integration responsibilities.

Integration constraint

The internal coefficient array is intentionally not reset. Reading an unwritten location or starting before every required coefficient is initialized produces undefined algorithm data. This implementation is not a drop-in standard-specific cryptographic NTT without external ordering, mapping, and known-answer validation.

3 APB Interface Background

3.1 Terminology

Setup phase The first cycle of an APB transfer, asserted with PSEL and PENABLE low.
Access phase The following cycle, asserted with PSEL and PENABLE high until PREADY completes the transfer.
Subordinate The IP receiving APB requests and returning PRDATA, PREADY, and PSLVERR.
Register offset Byte address relative to the IP APB base address.

3.2 Bus Transaction

An APB transaction starts with a setup phase and advances to an access phase on the next PCLK edge. Address, direction, and write data remain stable through completion. Read data and error status are sampled when PSEL, PENABLE, and PREADY are asserted together.

3.3 Bus Protocol

All APB and transform logic uses i_ntt_pclk. PREADY is permanently high, so every legal APB access completes without inserted wait states. Address validity and BUSY protection are evaluated when PSEL and PENABLE are both asserted. Only i_ntt_paddr[7:0] is decoded; integrations with a wider APB address must perform base-address selection externally. PRDATA is combinational during a selected read and is sampled on the completing access edge. PSLVERR is asserted only during the access phase.

4 Configuration Guide

4.1 Reset and Initialization

All sequential logic uses the rising edge of i_ntt_pclk and an active-low asynchronous reset i_ntt_presetn. Reset returns the controller to IDLE, clears live/sticky status, sets MODE to forward NTT, clears the coefficient index and datapath capture registers, and deasserts command/write pulses. The coefficient memory array itself is not reset. Reset deassertion shall be synchronized or otherwise guaranteed to meet the SoC reset methodology.

4.2 Clock Configuration

There is no programmable clock divider. PCLK directly clocks APB, controller, datapath registers, and coefficient memory. Maximum frequency is implementation-dependent because the combinational path contains multiplication, Barrett reduction, modular addition/subtraction, and twiddle selection. Static timing analysis shall close this path for the target process, voltage, temperature, and memory implementation.

4.3 Read and Write Registers

The four offsets 0x00, 0x04, 0x08, and 0x0C are valid. Other access-phase addresses return PSLVERR. STATUS writes are accepted but ignored. CTRL and COEFF_ADDR consume only byte strobe 0; a write without PSTRB[0] has no register effect. COEFF_DATA starts from the asynchronously read current coefficient, replaces each enabled byte, and commits the low COEFF_WIDTH bits only when at least one coefficient byte lane is enabled. Strobes above COEFF_WIDTH do not initiate a memory write. COEFF_ADDR remains readable and writable during BUSY, but every COEFF_DATA access and a CTRL write carrying START=1 during BUSY return PSLVERR. A non-START CTRL write during BUSY completes but does not update MODE.

4.4 Interrupts and Error Handling

PSLVERR is asserted for an invalid register address, a read or write of COEFF_DATA during BUSY, or a CTRL write with START asserted during BUSY. Starting inverse or pointwise mode is an accepted APB transfer but produces the internal ERROR pulse and sticky STATUS.ERROR. STATUS.DONE and STATUS.ERROR remain asserted until CTRL.CLEAR is written through byte lane 0. IRQ is the logical OR of those sticky bits. CLEAR has priority over a same-cycle status event.

5 Parameters

Table 5: Configurable design parameters

Parameter	Default	Description
C_APB_DATA_WIDTH	32	Implemented APB data width. Other values are not qualified by the fixed 32-bit internal register assembly.
C_APB_ADDR_WIDTH	8	APB address width; must be at least 8 because the low eight bits are decoded.
N	256	Transform length; must be a power of two and at least 4. The RTL does not elaborate safely for N=2 because several counters and twiddle indexes use LOG2_N-1 bits.
Q	3329	Modulus; must fit COEFF_WIDTH and support an element ROOT of exact order N.
ROOT	17	Forward twiddle root modulo Q.
COEFF_WIDTH	16	Byte-aligned coefficient and memory width, no greater than 32 and sufficient for Q. For the single-correction input reducer, require COEFF_WIDTH $\leq 2\lceil \log_2 Q \rceil$.
ADDR_WIDTH	8	Coefficient address width; must satisfy $2^{\text{ADDR\_WIDTH}} \geq N$.

6 Signals Description

6.1 APB Slave Interface Signals

Table 6: APB slave interface signals

Signal	Direction	Width	Description
i_ntt_pclk	Input	1	APB, transform, and coefficient-memory clock.
i_ntt_presetn	Input	1	Active-low asynchronous reset.
i_ntt_paddr	Input	8 default	Width is C_APB_ADDR_WIDTH; low eight address bits are decoded.
i_ntt_psel	Input	1	APB peripheral select.
i_ntt_penable	Input	1	APB access-phase qualifier.
i_ntt_pwrite	Input	1	High for write, low for read.
i_ntt_pwdata	Input	32 default	Width is C_APB_DATA_WIDTH.
i_ntt_pstrb	Input	4 default	One byte strobe per eight PWDATA bits.
o_ntt_prdata	Output	32 default	Width is C_APB_DATA_WIDTH; reserved fields read zero.
o_ntt_pready	Output	1	Transfer-ready indication, permanently high.
o_ntt_pslverr	Output	1	Invalid-address or protected-access response.

6.2 Interrupt Signals

Table 7: Interrupt interface signals

Signal	Direction	Width	Description
o_ntt_irq	Output	1	Level-sensitive request asserted while sticky DONE or ERROR is set.

7 Registers

7.1 Register Memory Map

Table 8: APB register map

Offset	Register	Access	Description
0x00	CTRL	R/W	Command and operating mode control.
0x04	STATUS	R	Live BUSY and sticky DONE/ERROR status.
0x08	COEFF_ADDR	R/W	Indexed coefficient memory address.
0x0C	COEFF_DATA	R/W	Coefficient value at the selected index.

7.2 Registers and Field Descriptions

7.2.1 CTRL

Name: APB NTT Control Register

Size: 4 bits

Address offset: 0x00

Software access: Read/Write

The reset read value is 0x00000002 because MODE resets to forward NTT. START and CLEAR are write-one pulses and always read as zero.

Table 9: *APB NTT Control Register Field Description*

Register Field	Bits	R/W	Description
Reserved	31:4	R	Reset 0. Read zero; writes ignored.
CLEAR	3	W1P	Reset 0. Clears sticky DONE and ERROR when PSTRB[0] is set.
MODE	2:1	R/W	Reset 01. Selects idle, forward NTT, inverse reserved, or pointwise reserved. Updates only while not BUSY.
START	0	W1P	Reset 0. Starts the selected operation. START during BUSY returns PSLVERR.

Table 10: CTRL mode encoding

MODE	Name	Behavior when START is written
00	Idle	No operation, no DONE, and no ERROR.
01	Forward NTT	Executes the implemented in-place radix-2 DIT transform.

MODE	Name	Behavior when START is written
10	Inverse NTT	Reserved; latches ERROR without modifying coefficient memory.
11	Pointwise	Reserved; latches ERROR without modifying coefficient memory.

7.2.2 STATUS

Name: APB NTT Status Register

Size: 3 bits

Address offset: 0x04

Software access: Read-only

Writes to this valid offset complete without PSLVERR but have no effect.

Table 11: *APB NTT Status Register Field Description*

Register Field	Bits	R/W	Description
Reserved	31:3	R	Reset 0. Read zero.
ERROR	2	R	Reset 0. Sticky unsupported-mode error; cleared by CTRL.CLEAR.
DONE	1	R	Reset 0. Sticky transform-complete status; cleared by CTRL.CLEAR.
BUSY	0	R	Reset 0. Live indication for INIT through NEXT controller states.

7.2.3 COEFF_ADDR

Name: APB NTT Coefficient Address Register

Size: ADDR_WIDTH bits

Address offset: 0x08

Software access: Read/Write

Let AW denote ADDR_WIDTH. The reset value is zero. The low AW bits select a location; upper write bits are ignored and upper read bits are zero. PSTRB[0] must be asserted to update the index. The index may be changed during BUSY, but COEFF_DATA remains protected. Values greater than or equal to N are outside the coefficient array and have undefined behavior rather than a defined PSLVERR response.

Table 12: *APB NTT Coefficient Address Register Field Description*

Register Field	Bits	R/W	Description
Reserved	31:AW	R	Reset 0. Read zero; writes ignored.
ADDR	AW-1:0	R/W	Reset 0. Selected coefficient index in the range 0 through N-1.

7.2.4 COEFF_DATA

Name: APB NTT Coefficient Data Register

Size: COEFF_WIDTH bits

Address offset: 0x0C

Software access: Read/Write

Let CW denote COEFF_WIDTH. This window accesses the memory location selected by COEFF_ADDR. Host readback is asynchronous when idle. A write performs byte merging with the stored word and occurs only when at least one low coefficient byte strobe is asserted. The memory reset value is undefined.

Table 13: *APB NTT Coefficient Data Register Field Description*

Register Field	Bits	R/W	Description
Reserved	31:CW	R	Reset 0. Read zero; write strobes above coefficient width are ignored.
DATA	CW-1:0	R/W	Reset X. Raw stored coefficient. Core capture reduces the value modulo Q.

8 Software Flow

8.1 Initialization Sequence

1. Apply i_ntt_presetn=0 while PCLK is present, then release reset according to the SoC reset strategy.
2. Write CTRL.CLEAR through byte lane 0 and confirm STATUS[2:0] is zero.
3. For each natural input index n, compute its $\log_2 N$-bit reversed index r.
4. Write r to COEFF_ADDR, then write $x[n]$ to COEFF_DATA with the coefficient byte strobes enabled.
5. Repeat until every coefficient location required by the transform is initialized.
6. Confirm STATUS.BUSY is zero before issuing START.

8.2 Normal Operation

1. Write CTRL with MODE=01 and START=1 using PSTRB[0]. For the default register encoding this value is 0x00000003.
2. Poll STATUS or wait for IRQ. Do not use BUSY deassertion alone as the completion event; wait for sticky DONE or ERROR.
3. If ERROR is set, follow the recovery sequence and do not consume the result as valid.
4. When DONE is set, write each desired index to COEFF_ADDR and read COEFF_DATA. With bit-reversed input loading, the result indexes are natural order.
5. Write CTRL.CLEAR, value 0x00000008 through byte lane 0, after the result has been accepted.

Completion observation

BUSY deasserts during the internal DONE state, while sticky DONE and IRQ are registered on the following PCLK edge. Software shall use STATUS.DONE or IRQ as the authoritative completion indication.

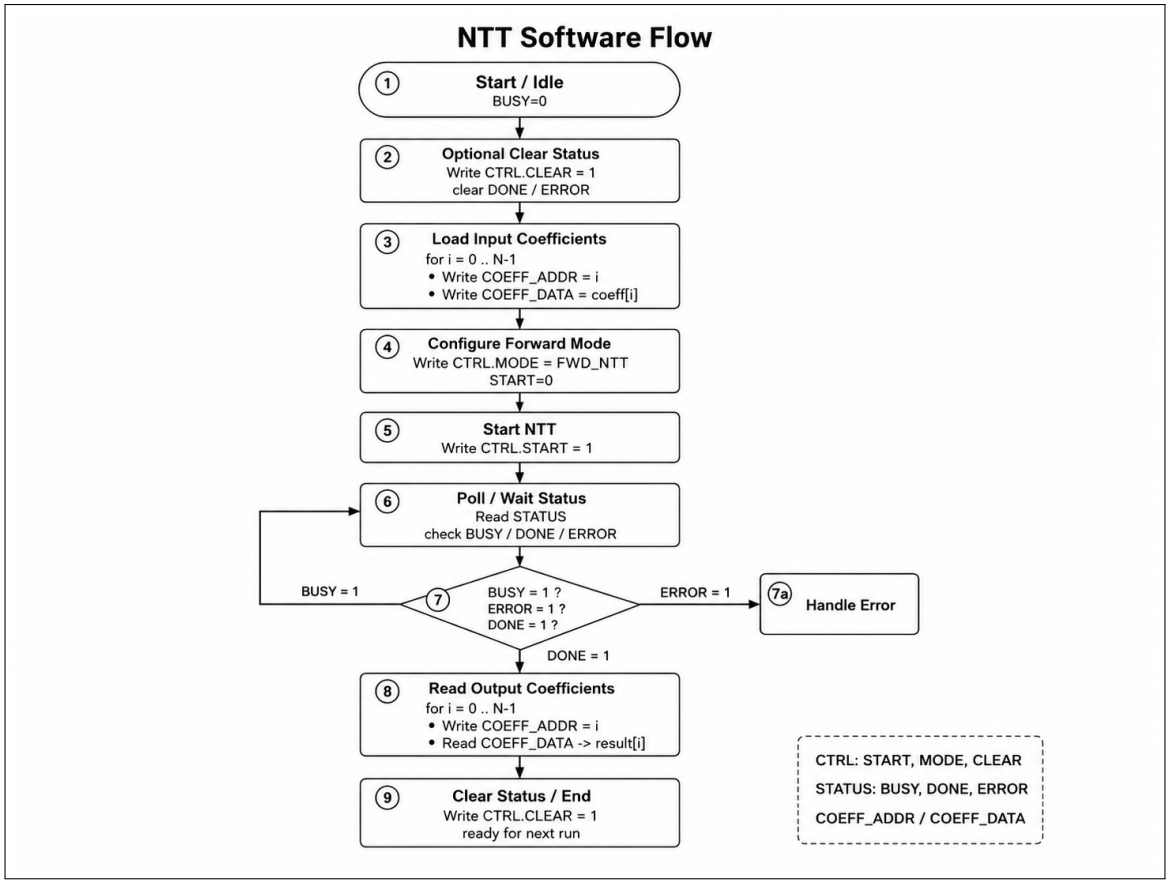

Figure 4: Recommended APB NTT software programming flow

Figure 4 summarizes the required register sequence. The coefficient-ordering contract remains

mandatory: the index written to COEFF_ADDR may be the bit-reversed form of the natural input index, as described in Section 2.3.

8.3 Error Recovery

1. On PSLVERR, stop the failing access and identify whether the address was invalid or the command/resource was protected by BUSY.
2. On STATUS.ERROR, record the attempted mode and write CTRL.CLEAR.
3. If a transform was interrupted by reset or its coefficient state is uncertain, reload all N coefficients before retrying.
4. Confirm BUSY=0, DONE=0, and ERROR=0, select MODE=01, and issue one START pulse.

9 Verification

9.1 Verification Environment

The reusable UVM environment verifies the APB programming model and IP-specific datapath behavior at the integration top.

9.1.1 Top Testbench

The top testbench generates PCLK and reset, instantiates the DUT and APB interface, publishes the virtual interface, and starts the selected UVM test.

9.1.2 APB Agent

The active APB agent contains the sequencer, driver, and monitor. Sequences describe software-visible transfers while the driver implements APB setup and access phases.

9.1.3 Monitor

The monitor reconstructs completed APB transactions and publishes them through an analysis port without driving DUT signals.

9.1.4 Scoreboard

The scoreboard maintains the reference programming model, checks responses and data, and samples functional coverage.

9.2 Verification Guide

The repository contains a UVM 1.2 environment around apb_ntt_wrapper. The verification top intentionally overrides the design to $N = 8$, $Q = 17$, ROOT=2, COEFF_WIDTH=8, and ADDR_WIDTH=3. This small legal configuration shortens the transform while retaining every controller state, all four software-visible registers, every coefficient index, and all MODE encodings.

The APB agent drives setup/access transfers and publishes completed transactions to the scoreboard. The scoreboard maintains an independent coefficient array and recomputes the forward transform from the stage, address, twiddle, and butterfly equations rather than sampling internal DUT results. The smoke sequence covers reset state, a complete transform, all result reads, and status clear. The coverage sequence adds invalid addresses, zero/partial/full PSTRB patterns, idle and unsupported modes, BUSY-time protected accesses, four coefficient patterns, sticky status behavior, and every coefficient index. If both configured tests complete, the scoreboard checks 40 coefficient reads in total: eight from the smoke test and 32 across four coverage-test transforms.

Table 14: Verification and static-analysis configuration status

Flow	Repository status	Defined scope
VCS RTL compile	Configured	Default parameters, ordered filelist.f, integration top apb_ntt_wrapper.
UVM smoke test	Implemented	One $N = 8$ forward transform, independent result comparison, DONE/IRQ clear.
UVM coverage test	Implemented	Four transforms plus APB address, strobe, mode, status, and BUSY protection cases.
VCS coverage gate	Configured	Line, condition, FSM, branch, and functional coverage; Makefile threshold is 90%.
SpyGlass lint	Project configured	RTL lint project and scoped waiver file are present.
SpyGlass CDC	Project configured	Single functional clock-domain analysis with project constraints.
SpyGlass RDC	Project configured	Asynchronous-reset structural analysis with project constraints and scoped waivers.

Sign-off status and limitation

No generated reports directory or tool-produced result artifacts were present in the reviewed source checkout. Therefore this specification records implemented tests and configured sign-off gates, not an independently confirmed PASS result or measured coverage percentage. Before product release, execute the flows in the target tool environment and archive their reports. Add a default-parameter $N = 256$, $Q = 3329$, ROOT=17 known-answer regression, explicit natural/bit-reversed ordering tests, reset-interruption cases, out-of-range coefficient-index tests, and implementation timing, power, and gate-level sign-off as required.

9.3 Coverage and Sign-Off

Sign-off evidence

Release evidence shall include compile, lint, CDC, RDC, simulation regression, functional coverage, and code coverage summaries generated from the tagged RTL revision.